Chapter 1

Introduction

The market penetration of smartphones is one of the fastest in technological history. In the U.S. market, the maturation (time from 10% to 40% market penetration) of smartphones was two times faster than other technologies such as the introduction of personal Internet access. In 2011, the U.S. market for smartphones could be considered as matured [27]. This phenomenon of rapid smartphone adoption could be observed globally. The Ericsson mobility report of 2016 [Eri16] estimates a world market penetration of 5.1 billion devices and expects an increase to 6.1 billion devices in 2022. Top adopters in 2016 were consumers in South Korea (88%) and Australia (77%) [Pou16]. **1.1**

Moreover, the software market volume for these kinds of devices and hardware is even much higher. In 2013, an average smartphone user has installed 26 mobile applications [49]. Thus, we calculate an overall market penetration of over 112 billion [Eri13] installed mobile applications in 2013. Besides, the research company *StatCounter* [48] reports that in October 2016, for the first time in history, the majority (51.3%) of website requests were done through mobile devices. This indicates that a lot of web-based applications are used by mobile clients (e.g., web browsers). **1.2**

A special feature of smartphones is their ability to detect contexts (e.g., location), which also drives the popularity of mobile applications [Tan08, Chap. 8.2]. While most smartphone users always carry their devices with them (or wear them), mobile applications could detect contexts of different kinds and act in a situation-dependent way. Such context-aware mobile applications may provide information tailored to the user and situation context, which is generally known as *smart* behavior. Novel device features (e.g., sensors) support the sensing of contexts. Context-awareness allows a lot of new mobile applications in industrial, health-care, and social application scenarios. Unfortunately, technical context changes (e.g., loss of connection) may affect the reliably of mobile applications in a negative way. **1.3**

To sum up, mobile devices and mobile applications nowadays are just as important as traditional personal computer systems and standalone applications. In the near future, they will be the primary personal and professional computer systems. Context-awareness gives rise to new business cases. Lack of context-awareness can cause negative effects during mobile application operation. Hence, context-awareness of mobile applications is an important and relevant area for academic research. **1.4**

The rapid evolution of mobile computing creates a heterogeneous landscape of software and hardware platforms, versions [Gro+14], and architectural designs. Considering the market shares of the different software and hardware platforms over the years, no software platform or vendor has such a majority that the remaining platforms can be ignored. The consensus is that multiple platforms and devices need to be considered further while developing a mobile application. **1.5**

After selecting a platform (e.g., Android), the heterogeneity continues. For example, to reach at least 90% of Android users currently requires creating a mobile application supporting versions of the Android platform spanning from version 4.2 (API 17) to version 7.1 (API 25) [11]. In addition, individual device features are **1.6**

very heterogeneous. Sensor availability and screen sizes may further reduce the potential user group. For example, the previously mentioned share of 90% Android users will be reduced to approximately 11% when requiring a large or extra-large screen for the developed mobile application. Additionally, current software engineering approaches do not provide the automatic adaptation or translation of a platform-specific implementation (e.g., Java used by Android) to another platform (e.g., Objective-C/Swift used by iOS). Hence, software engineering technology and development processes of mobile applications are not yet able to keep pace with the sudden rise of mobile platforms and the related heterogeneity.

1.7 Thus, developing mobile applications requires a development process that is different in many ways from the development process of traditional client-server applications or rich-client applications running on a single workstation. This development process, especially the labor-intensive implementation tasks, must be repeated for multiple software platforms (e.g., Android, iOS) to cover different user groups acceptably. For example, approximately 60% of the mobile applications were developed across different platforms [Joo+13]. Additionally, the test efforts will also increase for any additionally supported platform, version, or device (if tested at all). To mitigate or avoid these multiple native implementations, cross-platform solutions are often used.

1.8 Many existing frameworks and tools for mobile application development focus on cross-platform development as opposed to native application development. A native application has a single codebase that is written for one specific target platform and device type while using a software development kit (SDK). The codebase is translated by the appropriate target compiler and the resulting binaries run only on devices having the target platform. Requiring additional device-specific sensors limits the distribution of the application to devices of the same type. Developers must create and maintain multiple codebases to support diverse software platforms or device types. Thus, an obvious approach is to reuse a single codebase as a specification of the software system for different platforms [Hei+12]. This approach is known as *cross-platform development*. Heitkötter et al. distinguish three approaches for implementing cross-platform development: (i) runtime-based approaches, (ii) cross-compiling approaches, and (iii) model-driven development approaches (cf. Allen et al. [All+10]).

1.9 First, *runtime-based approaches* use a runtime environment to execute platform-independent program code or translated programs. For example, the Java Virtual Machine (JVM) is a well-known runtime environment executing compiled programs, while web browsers are good examples of program interpreters, e.g., JavaScript/HTML (Hypertext Markup Language.)[1]

1.10 *Cross-compiling approaches* use a cross compiler in order to translate platform-independent program code into a platform-specific or native code. Usually, they do not translate Java (favored in Android) into Objective-C (favored in iOS) or vice versa (except [Pud10]), but rather use source languages allowing program specifications in a more declarative way. This approach requires cross compilers for each of the desired software platforms. Additionally, there is often no or little abstraction between the source language and the native target language.

1.11 The last approach is the *model-driven development approach*. The model-driven development approach translates domain-specific models with high abstraction into platform-specific native program code. This approach also requires a model compiler (M2C), i.e. code generator, for each of the desired software platforms.

1.12 Web-based applications are a very popular example for *runtime-based* cross-platform development. Web-based and native applications follow opposing architectural approaches: web-based applications follow the traditional client-server paradigm and are multi-platform capable (e.g., Android, iOS). They provide good transactional

[1] This usually requires back-end systems e.g., application servers with rich functionality.

support based on the underlying database management systems, but they generally lack offline capability and cannot fully access the devices' sensors/hardware.

Although web-based mobile applications are multi-platform capable, their development approach could not be considered as platform-independent. The issue with web-based mobile applications is that functionality (if any) is located mostly on the server-side. Thus, mobile web-based applications could be considered as mobile services rather than autonomous mobile applications. In contrast, native applications can work autonomously and exploit device features (e.g., sensors, memory) very well. They usually lack platform portability due to their nativeness and platform- and device-tailored implementation. In case that native applications act as interoperable mobile clients of a multi-user system, conflict-free operation cannot be guaranteed. Different system contexts (e.g., loss of connection) cause problems in conflict detection while mobile clients are offline. **1.13**

Apparently, it is a problem to combine the advantages of the architectural designs and get rid of the disadvantages. While initiating a mobile application development project, teams often have to make an architectural and software platform choice (e.g., heading either towards a native and standalone mobile application architecture in Android/iOS or a platform-independent web-based mobile application) [Puv+16] [Bre+14], which thereafter dominates or limits the development process. Hence, software vendors and mobile application developers are increasingly asking for a development process and tools that help them to tackle the problem of the software platform and device heterogeneity and unleash the domain-specific opportunities of mobile computing (e.g., context-awareness) at the same time. Their goal is to roll out native mobile applications of high quality on the one hand, but avoid the development and test effort for all desired software platforms on the other hand. The mobile applications to be developed should additionally support different contexts, which is a specific requirement of the mobile computing domain. **1.14**

This leads to several general questions that are addressed in this thesis: **1.15**

- What are the features of mobile applications and the different platforms?
- What is supported by current (platform-independent) development approaches for mobile applications? What is missing?
- How do a development approach and tooling that covers missing parts look like?
- How can mobile applications support different contexts and how can a development approach support this aim?
- Which architectures of mobile applications are appropriate with respect to the domain-specific requirements (e.g., context-awareness)?

In the following, we provide the motivation for this work. Then we point out the challenges of mobile application development. Subsequently, the goals, the approach, and the contributions of this work are presented. This contains references to already published work. Finally, the remainder of this thesis is outlined. **1.16**

1.1 Motivation

As sketched in the preceding section, the main problem faced while developing mobile applications is the heterogeneity of platforms. Hence, platform-independent development approaches for mobile application development are generally useful to tackle this problem. **1.17**

This thesis builds on the model-driven development approach. "Model-driven development is simply the notion that we can construct a model of a system that we can then transform into the real thing." (Mellor et al. [Mel+03]) In terms of **1.18**

mobile applications development, a model describes several aspects of a mobile application in an abstract manner, whereas the transformation creates a concrete implementation. This approach has many advantages (cf. den Haan [21] and Selic [Sel03]) compared to a traditional development process and fits well into the domain of mobile application development:

1.19 The model-driven development approach uses a (domain-specific) modeling language, expressing software systems on a higher abstraction level compared to the codebase; the development might be faster and thus more cost-effective. A modeling element might be transformed into several lines of code or other artifacts. In the same time span, modeling allows expressing more functionality than writing code traditionally. Moreover, in case that the model-driven development toolset contains multiple code generators for different platforms, the leverage effect will be even stronger.

1.20 Another advantage of the model-driven development approach is its focus on business aspects as opposed to technical aspects. A model-driven development process can potentially be applied by domain experts rather than only by technical experts. This separation of concerns and skills empowers domain experts while technical experts focus on the needed toolset for model-driven development.

1.21 An increased quality of the resulting mobile application could be ensured by the model-driven development approach. The quality of traditionally developed mobile applications depends on the overall and architectural design, the skills, and discipline of the mobile application developers, and the used test and review process. Model-driven development frameworks provide automatic model validation and model quality analysis. Generated native program code follows established design patterns and has a high quality by design.

1.22 The model-driven development approach also resonates well with the high market pressure in the mobile sector. Technical changes between platform versions (platform evolution) can be concealed by the code generators. Migration of existing mobile applications, i.e., models, to a higher version of the software platform is well supported. In particular, beginners with less experience in mobile application development have a lot of trouble to find adequate architectural designs which meet their requirements. In the worst-case scenario, the selected architectural design is contradictory to the requirement specification. The model-driven development approach relieves the mobile application developers at this point and provides an adequate architecture. Besides, the model-driven development approach enables changes in an agile way because the application model can be improved and refined during the development process.

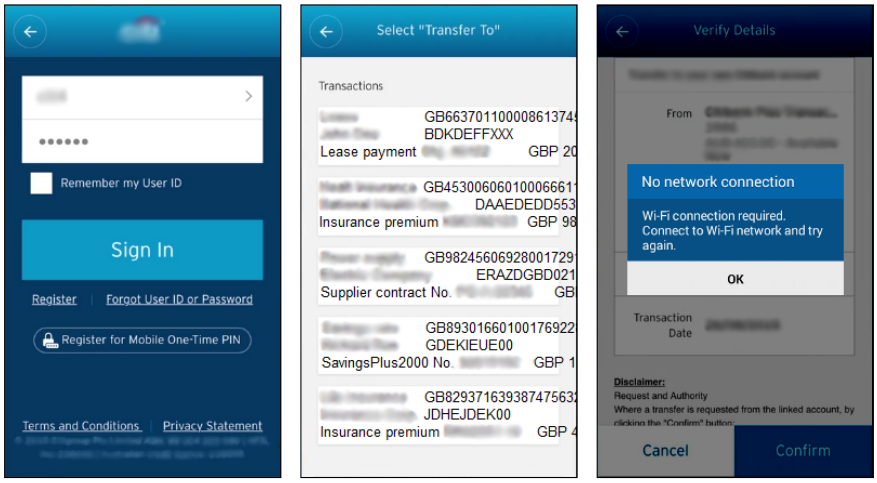
1.23 Finally, the model-driven development approach supports many use cases related to software engineering. For example, complex and error-prone program analysis tasks (e.g., concurrency problems such as deadlocks) on the level of code could be lifted to the model level [Sho+08]. Ideally, domain-specific model-driven development tools support mobile application developers while inspecting a problem of interest. With this support a manual review of the whole codebase to extract the relevant information will no longer be necessary.

1.24 To sum up, the model-driven development approach seems to be a promising approach in the domain of mobile application engineering, not only driven by the argument of multi-platform support. Although the model-driven development approach meets many software engineering requirements for mobile applications, a closer look at the different model-driven development approaches will reveal that there are further challenges.

1.2 Challenges

Several frameworks and tools that support model-driven development of mobile applications, such as Applause [Beh10] [05], MD^2 [Hei+13], JUSE4Android [SA14b], [SA14a], [Sil+14], Modagile [08], and Mobl [HV11b] [HV11a], have been presented in the literature. Their features will be discussed in detail later. With respect to these solutions, one might ask: why do we need an additional framework for the model-driven development of mobile applications? **1.25**

The answer is related to the genesis of mobile application development. The genesis of mobile application started with web-based applications that are executed on mobile devices. Web application developers usually deal neither with heterogeneous sets of platforms and devices nor with system contexts such as network interruptions. They assume a standardized platform (e.g., a web browser) without high variability of hardware or sensors and a stable network connection to the server. Accordingly, the mobile clients are minimal in terms of architectural design, while the servers are full-featured. Current model-driven development approaches for mobile applications adopt this concept: most frameworks and tools generate mobile applications that follow a client-server architecture. Without certain extensions, this architecture is generally insufficient to deal with changing system or device contexts. A look at the landscape of existing web-based mobile applications shows that the loss of network connection (system context) very often stops mobile applications unexpectedly. Additionally, the web-based mobile applications do not always provide an appropriate graphical user interface for the particular mobile device used (device context). **1.26**



(A) Login page (B) Insufficient scaling (transfer order templates) (C) Error message (transfer order)

FIGURE 1.1: Web-based mobile banking application

Example (Web-based banking software). Since online banking is the most popular way to make bank transactions, an increasing number of banking customers use mobile devices to access online banking services. Figure 1.1 shows a banking transaction order and the typical problems that may occur while using a web-based mobile application. A customer starts a banking session by entering the credentials at an initial login page (Figure 1.1a). S/he may have a list of transfer order templates from which one can be selected to make a money transfer. The first issue of this web-based mobile application is the inappropriate presentation of this data on the chosen device. Due to different device types, the web browser scales the graphical user interface to the devices's display size or density. As shown in Figure 1.1b, the text could thus be truncated. Besides, text may exceed the space inside the scaled **1.27**

widget elements. Hence, different device contexts are not fully supported by the shown web-based mobile application. Second, if the customer starts a transaction and meanwhile the network connection to the server is interrupted, the transaction will fail (Figure 1.1c). Even short interruptions will stop the transaction permanently because customers are usually required to log in again after every interruption of the network connection. A restart of the transaction is often not possible due to security reasons. Thus, web-based mobile applications support system contexts such as the connection context insufficiently. □

1.28 Dehlinger and Dixon [DD11] identified context-awareness as one of the major research directions for mobile applications software engineering. Consequently, a challenge of today's research in model-driven development for mobile applications lies in dealing with contexts. A context is the interrelation of a mobile application to other objects/subjects. These interrelated objects/subjects can be the *platform* on which the mobile application is deployed, the *device* on which the platform is operated, the mobile *user* which uses the mobile application, the different *system* environments (e.g., available/interrupted network service), and many others. Figure 1.2 shows the two main model-driven development approaches (Web-based approach and native approach) and the level of context-awareness they can reach within different contextual dimensions.

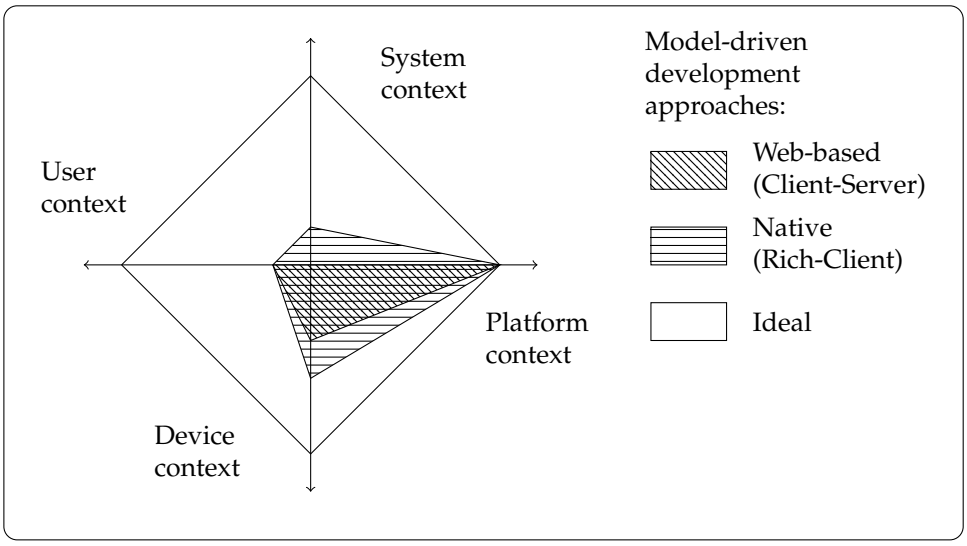


FIGURE 1.2: Model-driven development approaches and their contextual coverage

1.29 *Web-based* approaches are generally suitable for covering platform contexts. They also partly provide access to the features of a device. In turn, web-based approaches support system contexts such as the connection context poorly.

1.30 *Native* implementations cover device and system contexts in a better way. It should be noted that native approaches completely cover web-based approaches in terms of the contextual evaluation, but the effort to implement a native approach is much higher.

1.31 Web-based model-driven development approaches usually require one code generator, whereas a native model-driven development approach needs a code generator for every desired native platform. Hence, web-based model-driven development approaches also seem justified regarding development costs.

1.32 The advantages and disadvantages of native and web-based target languages are still subject of discussion [CL11].

1.33 Book et al. [Boo+05] discuss how the degree of mobility (which causes different system contexts) affects the architectural requirements. In turn, only an appropriate

architectural design of a mobile application enables context-awareness, particularly with respect to different system contexts. Several model-driven development frameworks do in fact support the generation of native program code; however, they follow a rich-client architecture (Applause [Beh10] [05]) or a client-server architecture (Modagile [08]). Thus, they are either only offline-capable or only online-capable.

To sum up, we face the challenge of context-awareness and propose a model-driven development approach to create mobile applications that can deal with different contextual dimensions. **1.34**

1.3 Goals of This Work

A fundamental goal of this work is to show that model-driven development can deal with the novel and emergent domain of mobile application development. Domain experts should be able to develop mobile applications quickly and easily with a model-driven development approach. Moreover, since the model-driven development approach is usually understood and implemented as a one-level approach, i.e., using only a design model to generate a static implementation, this work will demonstrate that a two-level approach comprising a design model and a runtime model is more flexible in terms of different contextual dimensions. Our generated native mobile applications contain both a static part and a part for interpreting runtime models. Hence, these mobile applications are no longer only static but can also be configured at runtime using runtime information. This hybrid architecture of the generated mobile applications supports different contexts, as demonstrated by four selected contextual dimensions, i.e., *platform*, *device*, *user*, and *system context*. The generated mobile applications are context-aware because the two-level model-driven development approach can handle these contexts very well. **1.35**

Moreover, an additional goal of this work is to provide architectural flexibility of the generated mobile applications depending on the type of mobile application. Using existing model-driven development frameworks for mobile applications, developers cannot change the architecture of the mobile application. The architecture is prescribed by the code generators of the used frameworks. Hence, the generated mobile application may not always meet the requirements (e.g., not online- and offline-capable, not interoperable with a back end system). Thus, an additional goal of this work is to provide different architectural designs, i.e., variants, for supporting different connection contexts, which are in turn part of the *system context*. These architectural variants enable different operation modes (only offline-capable, only online-capable, and online- and offline-capable) of the mobile applications. Additionally, a model-based simulation system (cf. [Bal+04]) supports mobile application modelers at design time to evaluate their design models by estimating the transactional throughput (number of successful transactions) for a particular architecture. **1.36**

1.4 Approach

Regarding Figure 1.2, we address the first three dimensions – *platform*, *device*, and *user context* – using a two-level model-driven development approach. This approach uses design-time models for code generation and runtime models to adapt already generated mobile applications at runtime. The approach is based on the generation of native mobile applications, as web-based solutions are generally unable to work offline and limited in hardware access. **1.37**

Supporting different *system contexts* (e.g., connection context) requires additional architectural features (cf. Book et al. [Boo+05]) as part of the generated mobile **1.38**

applications. However, mobile application modelers should not be involved in specifying the detailed architecture of the resulting mobile applications because we cannot assume that they are technical experts. Thus, mobile application modelers just declare the kind of the architecture (e.g., online- and offline-capable, only offline-capable, only online-capable), and the code generators configure the architectural components depending on the given requirements. The general architectural design is based on a generic extended client-server architecture supporting context-aware data and transaction management for mobile applications.

1.39 This overall approach has three major advantages. First, the model-driven development approach covers the *platform context* very well since different code generators can be used to generate platform-specific implementations from the platform-independent design models. Second, the flexibility of design models (e.g., through model transformation) allows the generation of *device*- and *user context*-specific versions of the mobile application. Moreover, runtime models enable some of this flexibility even during runtime. Finally, the approach makes domain experts capable of creating *system context*-aware mobile applications without requiring a special expertise in architectural design and the technical requirements. These advantages are evaluated by several case examples.

1.4.1 Design Models and Runtime Models

1.40 In order to support *platform*, *device*, and *user* contexts better, we propose a two-level modeling approach as a first fundamental difference from the existing one-level modeling approaches.

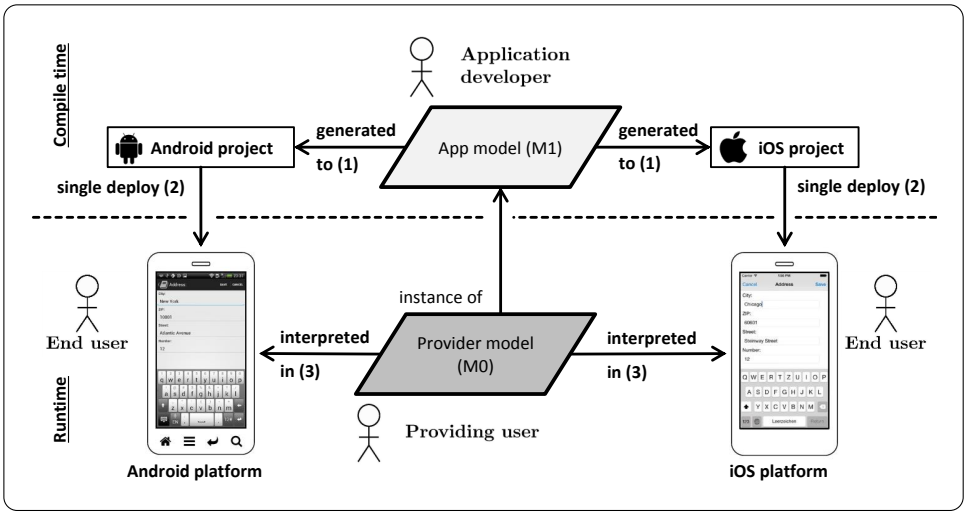


FIGURE 1.3: Two-level modeling approach

1.41 Figure 1.3 shows this novel approach in detail. A mobile *application developer* (not necessarily a technical expert) creates an *app model*. The *app model* conforms to a domain-specific modeling language (DSML) for mobile applications, which is developed during the course of this work. The mobile *application developer* can generate mobile application projects including source code and further artifacts (e.g., platform-specific graphical user interface descriptions, external libraries) for different platforms (Step 1). This already covers the *platform* context. After compiling these projects, a mobile application can be deployed to the mobile device of the respective platform (Step 2).

1.42 Using a traditional model-driven development approach, the deployed applications are usually not reconfigurable and must be remodeled, generated, and deployed

again in order to realize the modification. They could not provide many configurations for different user groups or platform types at the same time. Hence, our approach uses a second modeling level. A *providing user* may create one or more *provider model*s. A configuration given through this *provider model* is interpreted at runtime by the deployed mobile applications (Step 3). Hence, redeployment is not necessary. The *provider model* may modify the appearance, behavior, or data used in the mobile applications. This architecture implements a hybrid approach because runtime models are used as well as static application code [Ben+11]. The resulting applications provide runtime adaptability [Flo+06] but are not solely interpreted. This provides *user* and *device* contexts in an appropriate way, as shown later in detail by several case examples.

1.4.2 Extended Client-Server Architecture

As a second fundamental difference, we abandon the strict client-server and rich-client paradigm used by many existing model-driven development approaches. Based on the work of Satyamaran [Sat96] (cf. also Fuchß [Fuc09]), we follow an architectural paradigm called *blurred roles*. *Blurred roles* denote a relaxation of the commonly used client-server architecture. Parts of the server are duplicated on the client-side. **1.43**

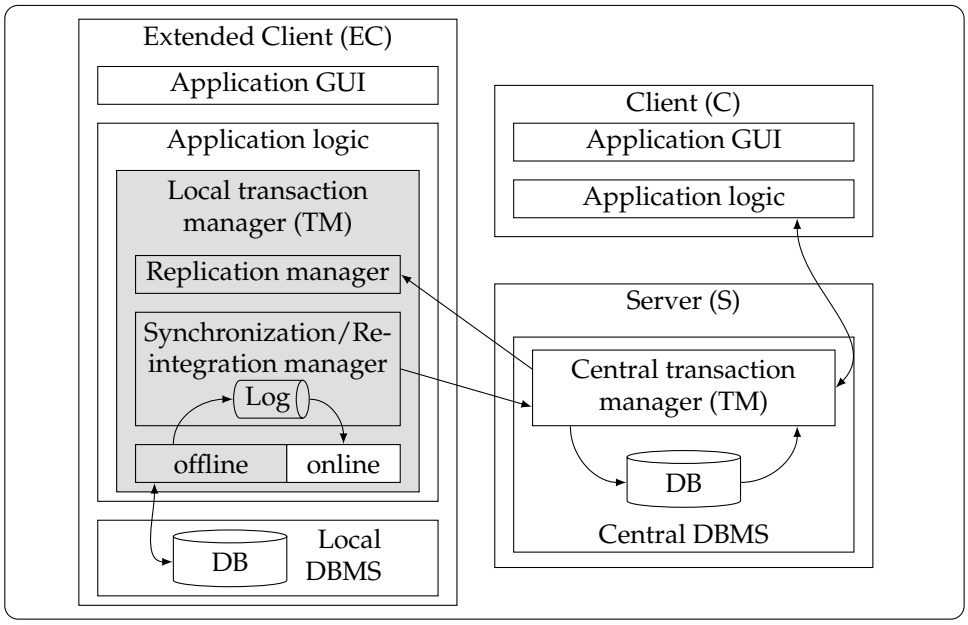


FIGURE 1.4: Extended client-server architecture vs. client-server architecture

A mobile application that follows the traditional client-server architecture, as is shown on the right-hand side of Figure 1.4, cannot operate when the connection of the client C and the server S is interrupted: required services, such as the database (DB), are no longer available. In turn, an extended client EC (left-hand side of Figure 1.4) can provide replicated data and transaction logging as part of a local transaction manager (TM). These facilities can be used while the extended client (EC) is disconnected from the server S. Moreover, the architecture of the extended client exploits a particular *mobile transaction model* (affects the implementation of the gray colored areas in the figure). Such *mobile transaction models* are required to avoid and resolve conflicts in multi-user environments. As we see later, the generated architecture of the mobile application plays an important role in increasing the transactional throughput of mobile applications. **1.44**

1.5 Contributions

1.45 The main contribution of this thesis is a framework for the model-driven development of mobile applications. Based on the particular design of this framework comprising more than state-of-the-art concepts, further contributions were made. These contributions deal with different contextual dimensions. The resulting main contributions are clustered in two parts: i) the model-driven development infrastructure and ii) the context support of mobile applications using this infrastructure. Moreover, the evaluation of each contribution constitutes an additional contribution of this work.

1.5.1 Model-Driven Development Infrastructure (Contribution 1)

1.46 The first main contribution is the model-driven development infrastructure itself. The development of the model-driven development infrastructure starts with a **requirements** elicitation and definition. The requirements are separated into modeling language requirements, architectural requirements, and tool requirements. A **domain analysis** of the target domain was subsequently carried out. We created a feature model describing the domain, even if this work does not support all identified features. After becoming more acquainted with the domain the model-driven development infrastructure should be designed for, we develop a **domain-specific modeling language** and corresponding graphical **model editor**s. This includes the creation of an abstract syntax (AS) and a graphical concrete syntax (GCS). Since a model-driven development infrastructure consists of both a domain-specific modeling language and **code generators**, the facilities for code generation are also a major contribution. In order to find appropriate architectural patterns of the mobile applications to be generated, we examined selected mobile applications and re-engineered **reference applications** that later serve as blueprints for the code templates and the architectural variants. This work supports the creation of platform-specific **code generators** [Vau+14] [Vau+18b] for Android and iOS. Such code generators for different software platforms support a platform-independent development process of mobile applications. Hence, the model-driven development infrastructure supports the dimension of platform context. Finally, based on the domain-specific modeling language and the code generators, several **prototypes** of mobile applications could be created. These contributions were developed while following an **agile bottom-up development process of domain-specific IDEs for model-driven development** [Vau+15]. The main components (i.e., the domain-specific modeling language and the code generators) of our model-driven development infrastructure are evaluated using different evaluation methods (i.e., design guidelines regarding the domain-specific modeling language, user experience evaluation, etc.).

1.5.2 Context Support for Mobile Applications (Contribution 2)

1.47 The second main contribution builds on the model-driven development infrastructure and deals with the support of different contextual dimensions. As in the first contribution, a list of **requirements** is given beforehand. The **domain analysis** is cycled again to come up with a feature model that respects additionally the context-related features of mobile applications.

1.48 Based on the two-level modeling approach (cf. Section 1.4.1), we implemented **user-** and **device context** support for mobile applications.

1.49 The model-driven development process and the generated mobile applications provide **design time instantiation** and **runtime instantiation** of application features (i.e., certain use cases) and provide manifold versions of mobile applications

[Vau+14] [Vau+16b]. In case of runtime instantiation, a regeneration or redeployment is not required. Mobile applications may be configured flexibly after deployment to certain user contexts. Hence, the model-driven development infrastructure supports the user context dimension. Moreover, considering the re-configurability of mobile applications in general, it might be interesting to also find more suitable mobile application variants related to their usability and ergonomics. For example, Lindner et al. [Lin+14] provide an efficient evaluation of the usability of Android applications by exploiting cognitive models.

The support of device contexts is also enabled by the two-level modeling approach. Particularly, a **design time adaptation** (model transformation) and **runtime adaptation** can adapt mobile application at design time or runtime to certain device contexts. Different device types can be supported by adapting the graphical user interface (GUI) automatically to different device contexts. This contribution supports also the device-specific adaptation of behavioral aspects of mobile applications. Hence, the model-driven development infrastructure supports the dimension of device context in many respects. **1.50**

Finally, to support different system contexts, we contribute a design process for the design of online- and offline-capable mobile applications. This design process is based on **model-based analysis**, **model-based simulation** , and **model-driven generation**. Moreover, the support of different system contexts requires modifying the code generators contributed in the first part of this thesis. In order to support online- and offline-capable mobile applications, the generated mobile application must follow the generic extended client-server architecture (cf. Section 1.4.2). A preliminary evaluation of the proposed design process is carried out. It shows that our design process is useful when it comes to evaluating app designs by their offline capability and that the mobile application developers can easily apply our design process. **1.51**

Apart from these contributions, the generated mobile applications provide features for **augmented reality** (AR), which is not discussed in this work [Guc+15] [Vau+18b]. **1.52**

1.6 Thesis-Related List of Publications

The following papers relate to the contributions of this thesis and have been published in publications which provide a scientific peer-review process (in chronological order): **1.53**

1. [Vau+14]: Steffen Vaupel, Gabriele Taentzer, Jan Peer Harries, Raphael Stroh, René Gerlach, and Michael Guckert. "Model-Driven Development of Mobile Applications Allowing Role-Driven Variants". In: *Model-Driven Engineering Languages and Systems - 17th International Conference, MODELS 2014, Valencia, Spain, September 28 - October 3, 2014. Proceedings*. Ed. by Jürgen Dingel, Wolfram Schulte, Isidro Ramos, Silvia Abrahão, and Emilio Insfrán. Vol. 8767. Lecture Notes in Computer Science. Springer, 2014, pp. 1–17

2. [Lin+14]: Stefan Lindner, Philippe Büttner, G Taenzer, Steffen Vaupel, and Nele Russwinkel. "Towards an efficient evaluation of the usability of android apps by cognitive models". In: *Proceedings 3. Interdisziplinärer Workshop Kognitive Systeme: Mensch, Teams, Systeme und Automaten, Magdeburg, 25.-27. März 2014*. Ed. by A. Wendemuth, M. Jipp, A. Kluge, and D. Söffker. Otto von Guericke Universität Magdeburg, 2014

3. [Guc+15]: Michael Guckert, Cornelius Malerczyk, René Gerlach, Gabriele Taentzer, Steffen Vaupel, and Michael Fatum. "Plattformunabhängige Entwicklung mobiler Anwendungen mit Augmented Reality-Funktionalität".

In: *Anwendungen und Konzepte in der Wirtschaftsinformatik – AKWI* 3 (2015), pp. 14–18

4. [Vau+15]: Steffen Vaupel, Daniel Strüber, Felix Rieger, and Gabriele Taentzer. "Agile Bottom-Up Development of Domain-Specific IDEs for Model-Driven Development". In: *Proceedings of the Workshop on Flexible Model Driven Engineering co-located with ACM/IEEE 18th International Conference on Model Driven Engineering Languages & Systems (MoDELS 2015), Ottawa, Canada, September 29, 2015.* Ed. by Davide Di Ruscio, Juan de Lara, and Alfonso Pierantonio. Vol. 1470. CEUR Workshop Proceedings. CEUR-WS.org, 2015

5. [Vau+16b]: Steffen Vaupel, Gabriele Taentzer, René Gerlach, and Michael Guckert. "Model-driven development of platform-independent mobile applications supporting role-based app variability". In: *Software Engineering 2016, Fachtagung des GI-Fachbereichs Softwaretechnik, 23.-26. Februar 2016, Wien, Österreich*. Ed. by Jens Knoop and Uwe Zdun. Vol. 252. LNI. GI, 2016, pp. 99–100

6. [Ger+16]: René Gerlach, Michael Guckert, Cornelius Malerczyk, Hans Christian Arlt, Steffen Vaupel, Gabriele Taentzer, and Michael Fatum. "Modellgetriebene Entwicklung mobiler Anwendungen mit Augmented Reality Funktionalität". In: *Mobile Anwendungen in Unternehmen*. Ed. by Thomas Barton, Christian Müller, and Christian Seel. Springer, 2016, pp. 193–211

7. [Vau+16a]: Steffen Vaupel, Damian Wlochowitz, and Gabriele Taentzer. "A generic architecture supporting context-aware data and transaction management for mobile applications". In: *Proceedings of the International Conference on Mobile Software Engineering and Systems, MOBILESoft '16, Austin, Texas, USA, May 14-22, 2016*. ACM, 2016, pp. 111–122

8. [TV16]: Gabriele Taentzer and Steffen Vaupel. "Model-Driven Development of Mobile Applications: Towards Context-Aware Apps of High Quality". In: *Proceedings of the International Workshop on Petri Nets and Software Engineering 2016, including the International Workshop on Biological Processes & Petri Nets 2016 co-located with the 37th International Conference on Application and Theory of Petri Nets and Concurrency Petri Nets 2016 and the 16th International Conference on Application of Concurrency to System Design ACSD 2016, Toruń, Poland, June 20-21, 2016.* Ed. by Lawrence Cabac, Lars Michael Kristensen, and Heiko Rölke. Vol. 1591. CEUR Workshop Proceedings. CEUR-WS.org, 2016, pp. 17–29

9. [Kri+17]: Lars Kristensen, Gabriele Taentzer, and Steffen Vaupel. "Towards Verification of Connection-Aware Transactions Models for Mobile Applications". In: *Petri Nets and Software Engineering. International Workshop, PNSE'17, Zaragoza, Spain, June 25-26, 2017. Proceedings*. Ed. by Daniel Moldt, Lawrence Cabac, and Heiko Rölke. Vol. 1846. CEUR Workshop Proceedings. CEUR-WS.org, 2017, pp. 227–228

10. [Vau+18b]: Steffen Vaupel, Gabriele Taentzer, René Gerlach, and Michael Guckert. "Model-driven development of mobile applications for Android and iOS supporting role-based app variability". In: *Software and System Modeling* 17.1 (2018), pp. 35–63

11. [Vau+18a]: Steffen Vaupel, Gabriele Taentzer, and Michael Guckert. "Model-Driven Design of Connectivity-Aware Mobile Applications". In: *Mobile Apps Engineering: Architecture, Design, Development and Testing*. Ed. by Ghita Kouadri Mostefaoui, Mitul Shukla, and Faisal Tariq. (To appear). CRC Press, 2018

1.7 Outline

The remainder of this thesis is structured as follows: the first part deals with the development of the model-driven development infrastructure for mobile applications. **1.54**

- The basic concepts and terms of the model-driven development approach and the foundations of mobile applications and their engineering will be presented in Chapter 2.
- Chapter 3 describes the chosen software development process model used while developing the framework for the model-driven development of mobile applications.
- Together with our domain experts, we collect the requirements of the model-driven development infrastructure and present these in Chapter 4.
- The domain analysis in Chapter 5 characterizes the target domain of mobile applications. As a result, a feature model describes the variants of mobile applications occurring in this domain.
- In Chapter 6, we describe the core of the model-driven development infrastructure, which is the domain-specific modeling language. We sketch the design decisions and define the domain-specific modeling language by a declarative metamodel. This metamodel is divided into data-, process-, and GUI sub-models. Besides, we provide well-formedness rules and graphical concrete syntax leading to the graphical model editor for app models. We also consider model quality. Finally, the domain-specific modeling language is evaluated with respect the modeling language requirements, existing guidelines for domain-specific modeling languages, and the guidelines for user interface description languages.
- Chapter 7 starts with the qualitative analysis of reference applications. We consider a mobile application that represents an information system and another mobile application that realizes a transactional system. A representative reference application was created subsequent to this analysis. Based on this representative reference application, the architectural design and the code can be abstracted and transferred to code generators. Additionally, the construction of code generators deals with the project and platform-specific initialization for the used IDEs, the architecture of the generated mobile applications, and the mapping of the model elements to platform-specific types. Subsequently, the processing of runtime models is explained. Finally, these contributions are evaluated.
- In Chapter 8, we present related work on model-driven development frameworks for mobile applications.

The second part of the thesis deals with the context support and demonstrates how the already provided or added functionality supports context-awareness. **1.55**

- The basic concepts and terms of context-awareness and the open issues will be presented in Chapter 9.
- In Chapter 10, we present the requirements related to the contexts of mobile applications.
- In Chapter 11, we continue the domain analysis to add context-related features to the feature model. As a result, the extend feature model describes, in addition, the properties of context-aware mobile applications.
- In Chapter 12, we show how the generated mobile applications deal with user contexts through design and runtime instantiation of role-specific processes. The evaluation includes two case examples.

- In Chapter 13, we explain how design time adaptation (e.g., model transformation) and runtime adaptation support different types of mobile devices. The mechanisms are demonstrated by a case example.
- Chapter 14 presents the design process for the design of online- and offline-capable mobile applications. To this, the model-driven analysis, model-based simulation, and model-driven generation are presented. These contributions are evaluated by an analysis of different app models. In the second part of the evaluation, we will report on the re-engineering of an existing real-world mobile application while using the proposed design process.
- Chapter 15 discusses the related work according to existing model-driven development frameworks for context-aware mobile applications.

1.56 In Chapter 16, we summarize this thesis. We give an outlook on possible future research related to this work.

1.57 This thesis contains also different appendices with further material: Appendix A (Well-Formedness Rules), Appendix B (App Models), Appendix C (Tutorial), and Appendix D (Miscellaneous).

Part I

Model-Driven Development of Mobile Applications

Chapter 2

Model-Driven Development and SE of Mobile Applications – Foundations and Definitions

The following sections exemplify the process and the components used in model-driven development approach. Since we are heading towards the construction of a model-driven development infrastructure in this work, we give a short introduction how these components can be designed. Subsequently, we present the state-of-the-art software engineering processes that are used to develop mobile applications and point out the open issues. Finally, we present several mobile platforms and their key concepts. **2.1**

2.1 Model-Driven Development

Compared to the document-centric and technology-dominated traditional software process models (e.g., V-Model [Boe79], Waterfall model [Roy87]), the model-driven development process [Mel+03] differs considerably. Model-driven development uses software models at all stages of the development process and is, therefore, a model-centric approach. This approach supports *maintainability* of software systems in such a way that modifications are made through modifications in the software models (cf. Lientz [Lie83], Lientz and Swanson [LS81] [LS80], and Lientz et al. [Lie+78]). *Portability* can be ensured by different code generators which generate native program code for different software platforms. In general, model-driven development increases the *abstraction* of software development and leaves most technical details up to the code generators. **2.2**

As stated before, the model-driven development approach uses a software model as the primary artifact. The term *model* is ambiguous, as it is used widely in general sciences and may differ considerably across different disciplines. Stachowiak [Sta73] defines that a model has the following properties: it represents an original (*mapping*), it reduces details of the original (*reduction*), and it has a purpose, i.e., is used instead of the original (*pragmatic*). Thus, models are usually *descriptive* in traditional scientific disciplines [Sei03]. However, Seidewitz states that models can be used to *specify* non-existing software systems, which is relevant to the model-driven development process. Thus, software models can be used both *prescriptively* and *descriptively* in the software engineering discipline. **2.3**

In this work, we adopt the term *model* from Kleppe et al. [Kle+03, p. 16]. They state "A model is a description of (part of) a system written in a well-defined language. A well-defined language is a language with well-defined form (syntax) and meaning (semantics), which is suitable for automated interpretation by a computer." This definition gives rise to the question to how a language can be defined well. **2.4**

In general, there are two fundamentally different [WK05] approaches to defining **2.5**

languages: the (i) grammarware and (ii) modelware approach.

2.6 The grammarware approach uses the Extended Backus-Naur Form (EBNF) as a metalanguage to define (programming) languages. *Graph grammars* are similar to Chomsky grammars which are used in formal language theory. The main idea of *graph grammars* is a rule-based transformation of graphs modeling and representing different kinds of systems (e.g., software systems) [Ehr+06]. Graph grammars will not be further pursued within this thesis.

2.7 In contrast, the modelware approach uses a meta-modeling approach in order to define *metamodels* describing again a set of models. We use the modelware approach in this work because a metamodel-based definition of languages provides several features, such as the creation of well-formed models, model transformation, model synthesis (code generation), and model comparison [Pai+12]. These features are essential for the model-driven development methodology.

2.8 *Metamodels* describe the permitted structure for models themselves. The terms *metamodel* and (domain-specific) *modeling language* will be used synonymously in this work because the domain-specific modeling language is defined by a metamodel. Moreover, we often will not differentiate between *modeling language*s and *programming languages*, but we are aware that *modeling language*s raise the level of abstraction and will, in general, not have the same expressiveness as *programming languages* (cf. Neubauer et al. [Neu+14]). The permitted structure of metamodels can be described again by metamodels, which is known as the *metaisation* [Str98] principle. Metamodels can be self-descriptive in order to avoid an endless *metaisation*. In this work, we follow a four-layered architecture (cf. Figure 2.1 left-hand side). The Object Management Group (OMG) standards Meta Object Facility (MOF) [34] and Unified Modeling Language (UML) [36]; [37] can be mapped (cf. Figure 2.1 right-hand side) to this four-layer architecture.

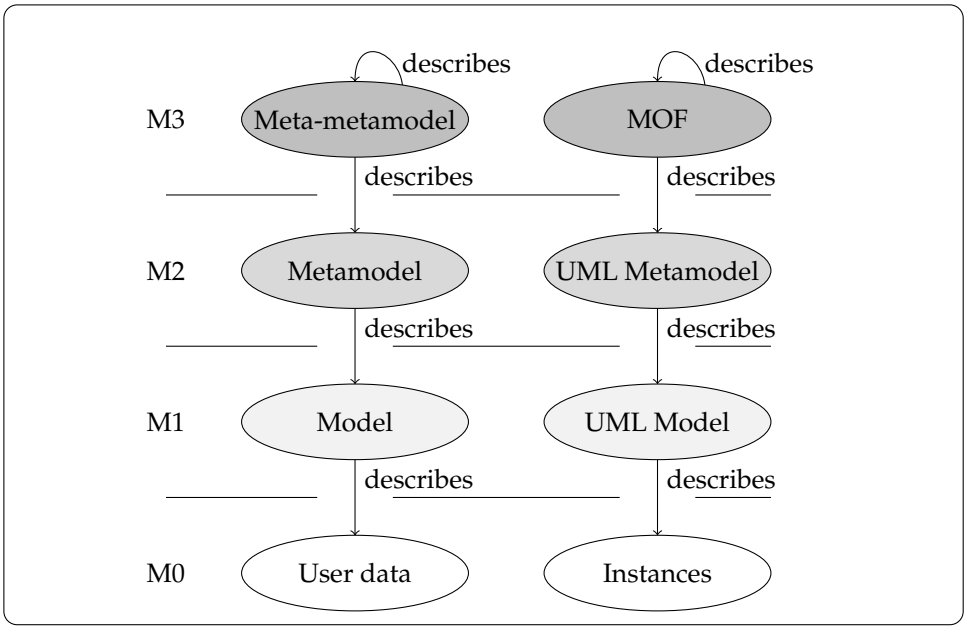


FIGURE 2.1: Four-layer metamodel architecture

2.9 The modelware approach covers the definition of an *abstract* syntax but disregards other aspects such as *concrete* syntax and *semantics* (cf. [Küh05]). A *concrete* syntax can be defined textually or visually [Fon07] by mapping the visual elements or textual tokens to the elements of the abstract syntax [Mul+08]. The concrete syntax may be derived automatically [Gar+06] from metamodels and kept synchronized [Rát+10] to the metamodel during changes. Since semantics specification is still a widely discussed topic [HR04] in the modeling community, we notice that the

semantic of a model and model elements are often specified informally in practice (e.g., in UML).

Authors such as Ludewig [Lud03], Aktinson [AK03], and Kühne [Küh05] presented more sophisticated definitions of *model*s and *metamodel*s that contain a discussion of linguistic and ontological aspects. **2.10**

Having defined the crucial artifact – the model – of the model-driven development process, we can turn toward the usage of the model. According to the taxonomy of model transformations provided by Mens and Van Gorp [MG06], model transformation can be *horizontal/vertical* and *exogenous/endogenous*. A horizontal model transformation will not lower/increase the abstraction level of the source and the target model, but a vertical transformation will do so (cf. Stachowiak's *reduction aspect*). Exogenous transformations have different source and target languages. Source and target language of an endogenous transformation are identical. Model refactoring is an example of an endogenous transformation. The model-driven development process shown in Figure 2.2 exemplifies different instantiations of these kinds of model transformations. **2.11**

The process starts with a computation-independent model (CIM). The vocabulary of the CIM follows the respective application domain (e.g., business) and contains no technical or computational aspects. The first model-to-model (M2M) transformation transforms the CIM into a platform-independent model (PIM). This kind of transformation can be classified as *exogenous* and *horizontal* transformation. The resulting PIM represents the structure and behavior of the specified software system. Instances of PIM models are usually UML models or other kinds of models which are used typically in the software engineering domain (e.g., Business Process Model and Notation – BPMN, Web Services Business Process Execution Language – (WS)-BPEL, and Systems Modeling Language – SysML). **2.12**

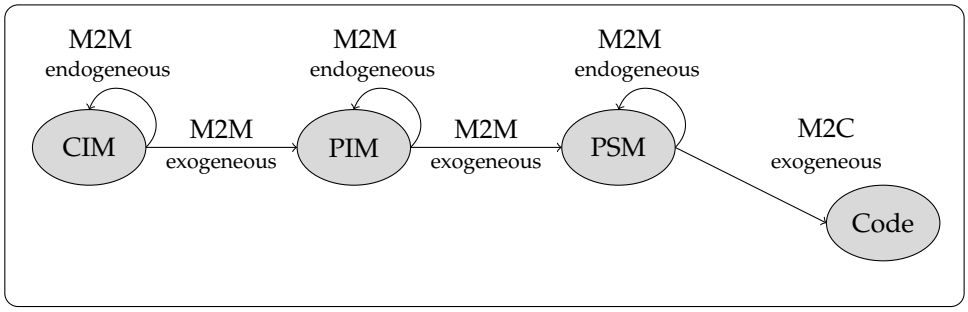


FIGURE 2.2: Common usage of software models in an MDD process

Using such models as a technical but platform-independent definition of the system, another M2M transformation creates the platform-specific model (PSM) as a result of an *exogenous* and *horizontal* transformation. A PSM expresses the structure and functionality provided by the PIM in a platform-specific modeling language. For example, a class model may define the data structure of a software system (as part of a PIM). The transformation from PIM to PSM may deliver a relational model, in case that a relational database is the target platform. While CIM and PIM are unique artifacts, a PSM may occur more than once, depending the number of platforms to be supported. **2.13**

The last transformational step transforms the PSM to native program code, which is called model-to-code transformation (M2C). This kind of transformation is an *exogenous* and *vertical* transformation. The generation of code is a *vertical* transformation because code generators usually add a lot of information and technical details and **2.14**

lower the level of abstraction. *Endogenous* and *horizontal* model-to-model transformations can be performed on all models (CIM, PIM, and PSM) and represent a refactoring of the model.

2.15 Based on the automation of the different model transformations, we will clarify that model-driven development is usually tool-supported and (semi)-automated. Domain specialists use a textual or graphical model editor providing a familiar vocabulary. The model editors also provide functions for validating and optimizing (e.g., evaluation and refactoring) created models. Quality-assured and validated CIM models are translated into PIM models. Technical specialists may add computation information to the PIM models using, again, model editors. The level of automation can be differentiated by the terms *-based* and *-driven*. Streitferdt et al. [Str+08] state that model-based development uses a model once for only one desired platform, while model-driven development reuses the model artifact in an automated process for several platforms.

2.16 As Figure 2.2 shows, the process is forward-oriented, i.e., the CIM will be transformed in a unidirectional way to the native program code. Consequently, modifications of the system are in general only possible by changing the initial artifact, the CIM. However, some frameworks [AC06] provide bidirectional transformations and keep the artifacts consistent irrespective of which artifact was modified. This concept is called *round trip engineering* (RTE). Indeed, RTE is not feasible for the model-driven development process as a whole, since it seems hard to recognize and map high-level constructs (e.g., model elements) to constructs of a lower level (code statements).

2.17 The final result of the model-driven development process is native program code, as Figure 2.2 shows. Further artifacts, such as program documentation and test cases can also be generated. Compiled native program code is executable on (native) runtime environments. In contrast to this model-to-code synthesis, other approaches directly execute platform-specific models on runtime environments [VG14]. This approach needs a runtime environment that can execute the PSM. Thus, we call the traditional model-to-code synthesizer a *model compiler* and the runtime approach a *model interpreter*. Each of these approaches offer advantages and disadvantages, respectively, which will be discussed in the next section.

2.18 One difference of the model-driven development process to the model-driven architecture (MDA) initiative [33] of the OMG is that the model-driven development process is generally not limited to the OMG standards (cf. Favre [Fav04]). The model-driven development process may contain domain-specific modeling languages which are developed for domains where they should be applied.

2.19 Based on the presented MDD techniques, we select the modelware approach to define our domain-specific modeling language. Our metamodel architecture has three layers, namely Metamodels, Models, and Instances. Moreover, our model-driven development infrastructure supports PIMs and PSMs.

2.2 Design of Model-Driven Development Infrastructures

2.20 In the previous section, we describe the artifacts and components (i.e., modeling languages and model editor; code generators) which are used inside a model-driven development infrastructure during the model-driven development of a software system. However, in order to introduce a model-driven development approach to a new domain, a corresponding model-driven development infrastructure must be first created by infrastructure developers. Although there is no comprehensive state-of-the-art process or automation for the systematic development of domain-specific model-driven development infrastructures, best practices have been proposed by several authors (e.g., by Kelly and Tolvanen [KT08, Pt. IV], Völter et al. [Völ+13, Pt. II], Völter [Völ09]). We want to clarify that these best practices do not describe

a general software development process on how model-driven development infrastructures could be created, but rather provide guidance on how individual components should be designed. Therefore, we contribute our agile bottom-up development process for domain-specific model-driven IDEs. Based on the following building blocks this process will be described in the next chapter.

2.2.1 Design of Modeling Languages and Model Editors

The hardest part during the development of a domain-specific model-driven development infrastructure may possibly be the identification of concepts, i.e., domain-specific language elements. Various authors, such as Deursen et al. [VD+00], Hudak [Hud96], Spinellis [Spi01], and Luoma et al. [TK05] carried out research pertaining to the domain-specific modeling language construction. They propose different methods for concept identification in new domains. **2.21**

Mernik et al. [Mer+05] describe a detailed approach to construct a domain-specific modeling language. This approach consists of the steps *decision*, *analysis*, *design*, and *implementation*. The *decision* motivates the development of the new domain-specific modeling language. The main reasons to develop a domain-specific modeling language are *notational improvement*, *task automation*, and *description of product lines*. Within the *analysis* step, the problem domain is identified and domain knowledge is gathered. Sources of knowledge are the technical documents, knowledge provided by domain experts (interviews), customer surveys and existing implementations, i.e., native program code in general-purpose languages (GPL). Mernik et al. point to two variation points within the *design* step. First, the designed domain-specific modeling language can exploit other modeling or programming languages. In this case, the domain-specific modeling language is embedded (piggyback) in another language. Otherwise, the domain-specific modeling language is developed and used in an isolated fashion (language invention). Second, the degree of formalism denotes whether the domain-specific language is described informally or with formal methods (e.g., grammars). Finally, the *implementation* steps describe how a domain-specific language can be implemented. Based on the assumption that models of a domain-specific modeling language are executable, Mernik et al. propose several implementation patterns for such languages. Three important implementation patterns are the *interpreter*, *compiler*, and *hybrid* approach, which are discussed in the next section in more detail. **2.22**

A domain-specific modeling language which is developed with the techniques mentioned before is not ready for use. The domain-specific modeling language needs a least one *concrete* syntax. Although the design of a textual or graphical concrete syntax is also a very creative part which affects the usability and acceptance of a modeling language in many respects, this task is supported well by tools. Many state-of-the-art frameworks support the semi-automated generation of textual and visual model editors. **2.23**

2.2.2 Design of Model Compilers

Technically, a code generator realizes a model-to-code (M2C) transformation. Since the input and output follow a well-defined language, the term *model compiler* is also often used in literature. Two alternative approaches are proposed in literature to create model compilers: **2.24**

2.2.2.1 Visitor-Based Approach

A *visitor-based* approach processes every model element and generates a stream of corresponding native program code. The framework Jamda (Java Model-Driven **2.25**

Architecture) [Boo03] [41] follows this approach. It provides the creation of metamodeling elements and corresponding Java classes and an application programming interface (API) for modifying these models. A visitor mechanism traverses the model and invokes code synthesis (CodeWriters) for the considered model element. The Jamda framework manipulates metamodel elements as long as possible by using cumulative transformations. A disadvantage of this framework is its focus on a top-down model compiler development. Having, as in our case, reference applications, it seems not feasible to decompose them into single transformation steps. Nevertheless, this approach will be applied partially to expand and decorate the input model with additional model elements (cf. Section 7.3.2) during a preprocessing step.

2.2.2.2 Template-Based Approach

2.26 Most model compiler frameworks (e.g., JET [67], FUUT-je, Codagen Architect, AndroMDA [24], ArcStyler, MetaEdit+ [TR03], and OptimalJ [Lon03]) follow a *template-based* approach. A template represents a unit of static code with meta-code *gaps*. These meta-code *gaps* are filled during the model compilation, using information from the app model as an instance of the domain-specific modeling language. Hence, template extraction is aimed at identifying which parts of a program are static, which parts are schematically recurring (e.g., declaration of attributes and corresponding getters and setters), and which parts depend on the modeled information. In contrast to the *visitor-based* approach, the *template-based* approach fits well to the bottom-up creation of a code generator.

2.27 Since the model-driven development infrastructure should generate runnable mobile applications, all related resources, i.e., layouts, mobile application project properties, and icons, must be generated as well, since the mentioned artifacts can also generated by a template-based approach. Hence, we deal not only with model-to-code transformation but also with a more general model-to-text (M2T) transformation. Actually, this would not affect the template-based approach. Both code generators to be developed are written in Xtend [Bet13] using the template-based approach.

2.28 **Example** (Native Program Code Template). Listing 2.1 shows a native program code template for the generation of POJOs (Plain Old Java Object). The main template function (lines 1-8) gets the modeling element *class* and generates the native program code of this class comprising the local field declarations and the accessor methods (getter and setter).

LISTING 2.1: Template for POJO generation

```
1  <<DEFINE Root FOR Class>>
2    public class <<name>> {
3      <<FOREACH attrs AS a>>
4        private <<a.type.name>> : <<a.name>>;
5      <<ENDFOREACH>>
6      <<EXPAND AccessorMethods FOREACH attribute>>
7    }
8  <<ENDDEFINE>>
9
10 <<DEFINE AccessorMethods FOR Attribute>>
11   public <<type.name>> get<<name.toFirstUpper>>(){
12     return this.<<name>>;
13   }
14   public void set<<name.toFirstUpper>>(<<type.name>> <<name>>)
       {
15     this.<<name>> = <<name>>;
16   }
17 <<ENDDEFINE>>
```

The first *static code* occurs in line 2 outside the meta-code tags («. . . »). This code will be generated without modification. The static code in line 4 may occur several times because it is part of the for-each-iteration. As seen in line 6, other template functions can be called again in order to generate native program code. This native program code appears at the location of template method invocation.

Using the class *Address* (cf. Figure 2.3) as input, the generated code is shown in Listing 2.2. From a developer's perspective, it is often not possible to distinguish between generated and manually written program code. We follow this attitude when generating human-readable code. Although models are the primary artifacts within the model-driven development approach, it is advisable to keep generated artifacts human-readable regarding labels and structure. **2.29**

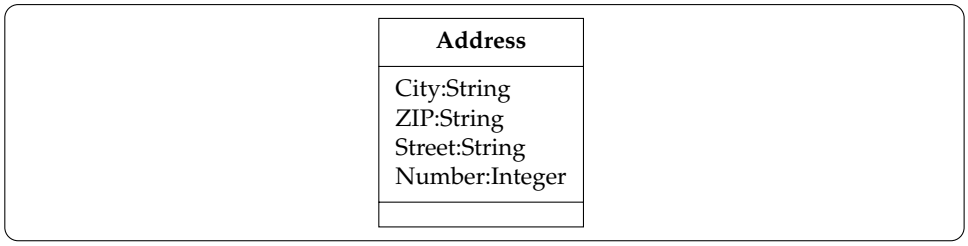


FIGURE 2.3: The class *Address*

LISTING 2.2: The compiled object *Address* (of type class)

```
1  public class Address
2      private String : City;
3      ...
4      private Integer : Number;
5
6      public String getCity(){
7          return this.City;
8      }
9      public String setCity(String City){
10         this.City = City;
11     }
12     ....
13 }
```

This encourages the manual extension of generated software prototypes (as required by our agile bottom-up development approach) as well as the maintainability of the code templates. □

A very common practice is the creation of code templates (cf. Listing 2.1) from existing code samples (cf. Listing 2.2) gained by reference applications. We will also follow this common practice of model compiler construction, but it should also be clarified how this approach is affected by model interpreter requirements. **2.30**

2.2.3 Model Compiler vs. Model Interpreter

As shown by the preceding example, the model compiler replaces the meta-code statements by the corresponding static information (e.g., class name, type names). The following two examples show the difference of a model compiling approach from a model interpreter approach: **2.31**

Example (Model compiler). Given the GUI model in Figure 2.4, the Page *ProcessesOverview* (ProcessSelectorPage) and its style settings can be translated into the native program code shown in Listing 2.3. **2.32**

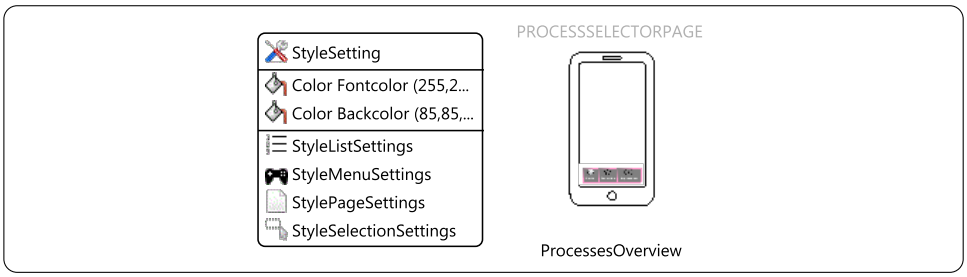


FIGURE 2.4: Excerpt of a GUI model (showing a *ProcessSelectorPage* and *StyleSettings*)

LISTING 2.3: The compiled object *ProcessesOverview* (excerpt)

```
1  ...
2  public class MainProcessesActivity extends Activity {
3     ...
4     private void createProcessList() {
5        ...
6        main_list.setFontColor(...(255,255,255));
7        ...
8     }
9  }
```

2.33 The corresponding code template evaluates the meta-code statement «`styleSetting.getFontColor().getRed()`» and generates the specified color value (=255) to the native program code in line 6. A program-code compiler processes the generated native program code, and the resulting mobile application shows the modeled behavior, i.e., the specified font color. □

2.34 Assume now the installation of the generated mobile application on different devices. For example, the mobile application is deployed to an eBook reader that only provides a monochrome color scheme. The mobile application may not be usable because of the unfavorable style scheme. A common practice when using the model-driven development approach is to modify the GUI model and to generate a new mobile application variant that fits better to the desired device type. By following this approach, to deal with the device heterogeneity, a countless set of mobile applications and app models will result. According to the proposed approach (cf. Section 1.4.1) to use *design models* and *runtime models*, a better solution seems to be the configuration of some values at runtime. This is what a model interpreter realizes.

2.35 **Example** (Model interpreter). Given the same setting as in the preceding example, a slightly changed code template produces the native program code shown in Listing 2.4. The code template generates static code that works similarly to the meta-code statement shown in the preceding example.

LISTING 2.4: The partly compiled object *ProcessesOverview* (excerpt)

```
1  ...
2  public class MainProcessesActivity extends Activity {
3     ...
4     private void createProcessList() {
5        ...
6        main_list.setFontColor(...(
7           RuntimeModel.getStyleSetting().getFontColor().getRed(),
8              ...));
9        ...
10    }
11 }
```

The introduced statement in line 7 loads a runtime model at runtime (provided by the class `RuntimeModel`) and inserts the corresponding color value. Thus, the resulting mobile application can be configured at runtime by a runtime model. Modification of the design model, regeneration of the mobile application, and redeployment are not necessary. □

2.36 The question that may arise now is: why construct a model compiler instead of a model interpreter? Similar to the traditional discussion regarding interpreters and compilers, both have advantages and disadvantages.

2.37 Model compiling has the following advantages (cf. den Haan et al.[1]) over model interpretation:

2.38 Model compilers capture a lot of technical domain knowledge in the shape of generation rules. The generated mobile applications can be generated in such a way as to look like manually coded mobile applications. Model instances and generation rules remain hidden to the mobile end user. Thus, compiled model-based mobile applications do *not reveal too much knowledge* about the domain and are *easier to understand* compared to a model interpreter.

2.39 From an infrastructure developer's perspective, a model compiler approach is *easier to start with* because reference applications can be used for code extraction and concept analysis. Besides, the model compiler approach is more *iterative*. A number of meta-code statements inside a code template could be increased or refined during the creation of the model-driven development infrastructure. Another great advantage consists in *debugging* support. The generated native program code can be easily debugged using conventional debugging approaches, while model interpreters are hard to debug.

2.40 In contrast, model interpreters have the following advantages over model compilers:

2.41 Model interpreters enable *changes at runtime*, which is of great importance regarding the issue of context-awareness. Regarding platform portability, model interpreters may abstract from concrete platforms and execute platform-independent models. Finally, a model interpreter usually needs to be deployed only once and can then be used for runtime modifications of the mobile application using the runtime model.

2.42 Although the model interpreter approach sounds very flexible, not all platforms fully support such flexible instantiation. For example, the Android platform does not support the programmatic access to all graphical user interface elements due to a static resource management. Thus, the implementation of the model interpreter is sometimes limited by the runtime behavior of a software platform.

2.43 Based on the presented design variants for the different components of a model-driven development infrastructure, we developed a domain-specific modeling language as a self-contained language that depends on no other language. We initially focus on a graphical concrete syntax for this language, even if a concrete textual syntax might be useful for certain groups of users (i.e., technically-skilled developers). The code generators follow a template-based approach since most meta-tools support this kind of approach. Moreover, a template-based approach allows a bottom-up construction of code generators. Finally, we followed both a model compiler approach and a model interpreter approach in this thesis, particularly to benefit from the advantages of each of the approaches. Later on, we will describe which information can be modeled and evaluated at runtime (cf. Section 7.3.4).

2.3 Software Engineering of Mobile Applications

2.44 Next, we turn our attention to the software engineering process of mobile applica-

[1] http://www.theenterprisearchitect.eu/blog/2010/06/28

tion. Empirical studies reveal that software engineering of mobile applications is dominated by agile methods, as 86% of survey participants confirm that the agile methods are appropriate for the development of mobile applications [Flo+14a]. Scrum was the most closely followed approach. Apart from these empirical findings, Abrahamsson shows that agile methods fit very well to the traits observed in mobile software development [Abr05]. Agile methods match well because of small development teams (41% have 5-15 members [Flo+14b]), short development cycle durations (57% projects have a project duration of 6-18 months [Flo+14b]), and high product release frequency (46% of the projects have a monthly release), whereas traditional software engineering processes need more lead time for up-front analysis and design.

2.45 Although general agile methods can be applied, the existing agile methods were refined in order to optimize them for the particular requirements of mobile application development. Proposed approaches are Mobile-D [Abr+04] from Abrahamsson et al., Hybrid Method Engineering (HME) [RR08] by Rahimian and Ramsin, the Mobile Application Software Agile Methodology (MASAM) [Jeo+08] by Jeong et al., and the Scrum and Lean Six Sigma (SLeSS) approach [Cun+11] by Cunha et al. In this context, the work from Rahimian and Ramsin is noteworthy because they map the main characteristics of agile software development to the traits observed in mobile software development. Hence, agile principles and mobile application development match well. Unfortunately, most of these development processes have not been evaluated or tested in practice.

2.46 The community still considers a number of open issues: in 2007, a GI[2] Dagstuhl Research Seminar on *Software Development Methodologies for Mobile Applications* defined three main challenges of mobile application software engineering. The main research dimensions identified by the participants are i) the *architecture* of the device, ii) the *data and context management*, and finally iii) the *user interface heterogeneity* [Kön09]. Wasserman [Was10] confirms the mentioned issues but also explains that *non-functional requirements*, such as battery lifetime or exceptional events such as the loss of connectivity, are yet to be covered well by current software engineering approaches. He also mentions *portability* as an open issue and sketches different research directions to solve the problem. Additionally, Dellinger and Dixon [DD11] propose the use of self-adaptive mechanisms to better support the dynamism in mobile applications at runtime. Muccini et al. [Muc+12] highlight the insufficient methods to *test* mobile applications. Besides these well-separated research directions, Alencar and Cowan [Ale12] point to several additional issues and emergent applications.

2.3.1 Model-Driven Development of Mobile Applications

2.47 As already shown in Figure 1.2, the existing model-driven development frameworks show only a low contextual coverage, even if the generated mobile applications follow a web-based or native implementation. The state-of-the-art of model-driven development of mobile applications has two major shortcomings: first, the traditional model compiler approach hardly allows any configuration of the mobile applications at runtime. Consequently, a mobile application cannot be configured properly to a specific device- or user context. Second, the architecture of the generated mobile applications is defined by the code generators. Hence, the architecture cannot be changed or modified according to the current requirements of the mobile applications.

[2] German Association of Computer Science

2.4 Mobile Software Platforms

This section presents the state-of-the-art concepts and technologies that are provided or used by the major mobile software platforms. We select two native platforms (Android and iOS) for this presentation since they are the most common mobile platforms. Additionally, we present a cross-platform technology, since the cross-platform approach is (due to its platform-independence) closely related to the model-driven development approach. We take a closer look at Apache Cordova [03] since it is a representative state-of-the-art framework for cross-platform development of mobile applications. Apache Cordova is available for many software platforms. **2.48**

2.4.1 Android – Concepts and Technology

The first commercial version of the Android operating system was released in 2008 by the Open Handset Alliance and Google. The operating system is open-source (Apache License 2.0/GNU GPL v2 for the Linux kernel), but may have proprietary third-party components. It targets smartphones, tablet computers, TVs, eBooks and different consumer electronics devices from a huge set of different vendors. It runs on 32/64 bit ARM (Advanced RISC Machines), x86, x86-64, MIPS and MIPS 64 (MIPS Technologies) hardware architectures. A graphical multi-touch user interface serves as the default user interface. **2.49**

The internal software stack of the Android operating system (see Figure 2.5) is based on a *Linux kernel*. The kernel provides process and memory management and connects the hardware of the device to the application-oriented upper layers. Several libraries inside the *Libraries* layer provide, for example, HTML rendering (WebKit), persistence (SQLite), C-language support (libc), security functionality (Secure Sockets Layer – SSL), media/graphics support (Open Graphics Library for Embedded Systems – OpenGL/ES; Skia Graphics Library – SGL), and other built-in functionality. The sub-layer *Android Runtime* is particularly relevant in the context of our work, as it used to execute Java-based programs. Its Dalvik Virtual Machine (DVM) is the counterpart to the Java Virtual Machine (JVM). The core libraries provide different Android-specific classes and Java core classes (e.g., `android.*`, `java.*`, and `javax.*`). However, this library does not contain all standard Java SDK classes. The Dalvik virtual machine is designed for limited resources (e.g., CPU, RAM avg. 10MB, max. 64 MB) and works based on registers (instead of using a stack). The *Application Framework* layer provides many higher-level services to Android applications. Finally, the *Applications* layer comprises the installed mobile applications. This includes standard applications (e.g., Home Screen, Browsers, and E-Mail reader) and user-installed applications. The latter ones are distributed over a mobile application store. Mobile applications which are to be distributed through this store must be signed by the publishing authority. **2.50**

From a mobile application developers' perspective, an Android application is comprised of Java executable files (in Dalvik format), XML (EXtensible Markup Language) files that describe resources and layouts, and different mobile application resources like videos, images, and audios. All artifacts are bundled to an .apk-File (Android Application Package – APK). An essential file of this package is the Manifest file, which contains meta-information about the developed application. The APK contains, amongst standard Java SDK classes, classes which are part of the Android SDK library. This library reflects the application framework layer shown in Figure 2.5. For example, *Activities* can be used to create a screen that can interact with the user. In turn, *Activities* can interact with each other using *Intents*. Functionality that has no user interaction through the graphical user interface can be provided by non-visible *Services*. As an example, a music player might be realized with a service. Since every mobile application runs in its own instance of **2.51**

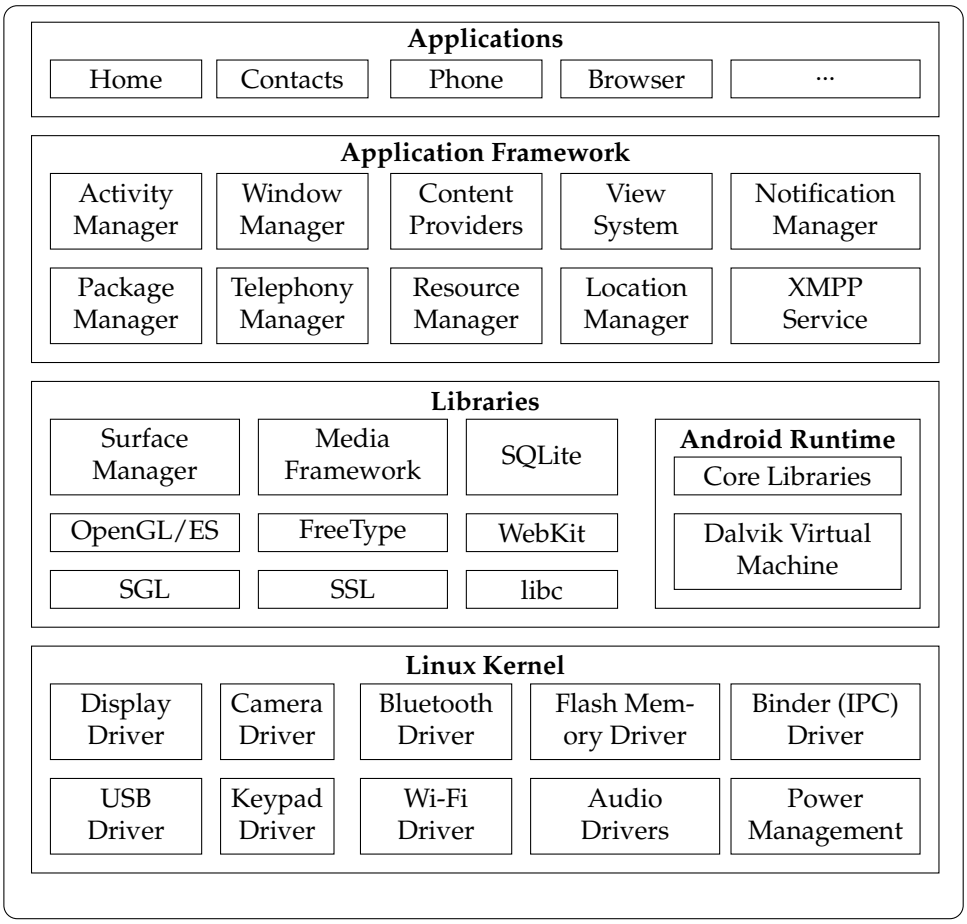


FIGURE 2.5: Android software stack

the virtual machine, *Content Providers* and *Broadcast Receivers* are used to establish a communication between mobile applications.

2.52 A model-driven development infrastructure designer must keep in mind that the Android platform neither provides full support in terms of the Java SE specification nor the Java JEE libraries. Any functionality which is taken from a third party library must be bundled to the developed mobile application. Moreover, the Dalvik virtual machine is also limited in many respects (e.g., the overall number of methods in a mobile application).

2.4.2 iOS – Concepts and Technology

2.53 The first version of iOS (formerly known as iPhone OS) was released in 2007 by Apple Inc. The operating system is closed-source. It exclusively targets devices from Apple, such as smartphones (iPhone), tablets (iPad), and multimedia devices (iPod Touch). It runs on different versions of ARM hardware architectures. The user interface is graphical and multi-touch capable.

2.54 The internal software stack of iOS (see Figure 2.6) is based on the XnU[3] kernel which is, in turn, an incarnation of the Mach kernel (Carnegie Mellon University). Moreover, the Darwin operating system and Berkeley Software Distribution (BSD) are also ancestors of iOS. The *Kernel and Device Drivers* layer provides additionally the device drivers. The *Core OS* layer provides frameworks for realizing process intensive calculations (Accelerate Framework), interaction with wireless devices

[3] X is not UNIX

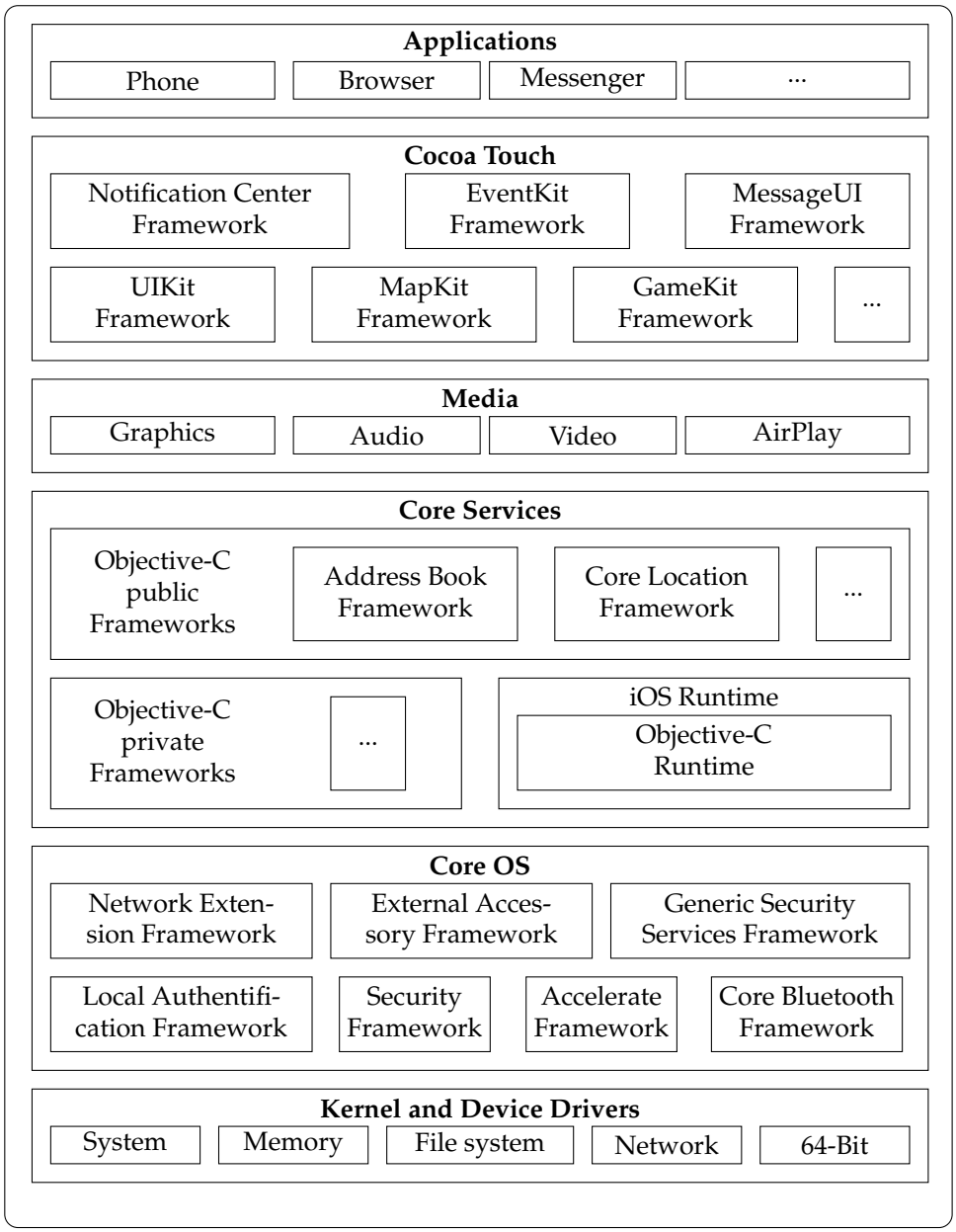


FIGURE 2.6: iOS software stack

(Core Bluetooth Framework), accessing external devices which are connected via the docking interface (External Accessory Framework), secure applications (Generic Security Services Framework/Security Framework), local authentication (Local Authentication Framework), and connections and configurations (Network Extension Framework) of Virtual Private Networks (VPN). The *Core Services* layer provides the Objective-C runtime and the Objective-C private and public frameworks. The public frameworks can be accessed by every third-party application while the private frameworks are reserved for manufacturer applications, i.e., Apple's own applications. The *Media* layer provides many frameworks for processing media of different kinds. The *Cocoa Touch* layer contains the key frameworks for building iOS applications. These frameworks (e.g., the UIKit Framework) provide the basic application infrastructure such as user interface management, the model-view-controller stubs, graphics and windowing support, etc. Finally, the *Applications* layer contains the pre-installed mobile applications as well as the user-installed applications. The

latter ones are distributed over a mobile application store.

2.55 From a mobile application developers' perspective, an iOS application contains compiled and executable files (prepared for ARM architecture), NIB files (NeXT Interface Builder) which contain the layouts, and resources like videos, images, and audios. All artifacts are bundled to an .app file (Application bundle file) which in turn is enveloped by an .ipa-File (iOS application archive file). An essential file of this archive is the Info file (Info.plist), which contains the meta-information of the particular mobile application. The archive may contain standard Objective-C classes which are not specific to iOS, as well as classes which relate to the frameworks shown in Figure 2.6. Typical iOS-specific classes are *UIViewController* which are used to interact with the user through the graphical user interface. A transition between view controllers can be realized with *Seque*s.

2.56 A model-driven development infrastructure designer must keep in mind that the generation and code compilation of iOS applications requires the proprietary XCode building tool in order to seal and encrypt the resulting binary. Moreover, the set-up of a project structure inside or outside XCode might be not supported as XCode is a proprietary IDE.

2.4.3 Comparing Android and iOS

2.57 As shown in Figures 2.5 and 2.6, the architectures of both platforms follow a layered architecture containing similar components. The biggest difference according to the software platform is the underlying hardware platform. iOS is limited to proprietary devices (e.g., iPhone and iPad), whereas Android can be found on devices manufactured by different vendors.

2.58 From a software developer's perspective, both platforms follow a declarative editor-based definition of the graphical user interface (XCode - iOS/XML - Android). Business logic can be specified in commonly-used programming languages (Objective-C – iOS/Java and C – Android). However, mobile applications for iOS can only be developed with the XCode IDE, which runs only on macOS (Apple). In turn, different IDEs (Eclipse, Android Studio) for the development of Android applications are available for different operating systems (e.g., macOS, Windows, and Linux). Since both platforms generally follow the same concepts, the market shows no clear leader.

2.59 Hence, the selection of a native platform depends more on individual factors (e.g., personal technical skills, license budget, and targeted user group). However, Android is often considered as more cost-effective and used widely in an academic context. For more details, please refer to Goadrich & Rogers [GR11] and Gronli et al. [Grø+10].

2.4.4 Cross-Platform Technologies

2.60 As shown in the introduction, the cross-platform approach follows the credo "write once, run everywhere" whereas the model-driven development approach sound "model once, run everywhere". These statements show that both approaches are very strong competitors and mobile application developers, especially technical experts, often have difficulties to choose between these approaches. Hence, we also present the concepts of cross-platform development based on a current cross-platform development framework. Additionally, one not pursued possibility of the model-driven development approach is the generation of cross-platform program code. Indeed, model-driven development could cover the cross-platform approach in case that the model-driven development infrastructure provides a code generator for cross-platform languages. The implementation of a cross-platform code generator is advised, provided high platform coverage is pursued but a moderate

hardware access can be accepted. Thus, the implementation of one cross-platform code generator could be more cost-effective than the implementation of one or more native code generators.

A popular, state-of-the-art cross-platform technology is *Apache Cordova* [03] (formerly PhoneGap). Apache Cordova uses CSS3 (Cascading Style Sheets – Level 3), HTML5 (Hypertext Markup Language Version 5) and JavaScript as a platform-independent programming language. The framework supports different platforms, such as Android, iOS, Blackberry, and Windows Phone. An Apache Cordova application contains HTML5 and JavaScript. It can make calls to native application code of the particular platform it is running on. Thus it follows both a runtime approach by executing the web-based parts and a cross-compiling approach by providing the ability to execute native application parts. Native routines must be implemented beforehand on all targeted platforms by the IDE developers. These functions are stored in API libraries. Mobile application developers that need further functionality may create their own plugins. Similar to the traditional development approach, these plugins must be manually implemented for each target platform (e.g., iOS and Android). **2.61**

Figure 2.7 shows the architecture of an Apache Cordova application running on the Android platform. The implementation looks similar for other software platforms. The application is separated into a *Web Architecture* part (left-hand side) and an *Android Cordova Container* (right-hand side). The *HTML Android Application* container contains the developed application, which is separated into a *UI Layer* (HTML, CSS, and JavaScript; cf. Oehlman and Blanc [OB11]) and an *Application Logic* layer. The *HTML Android Application* makes HTML/JavaScript and Cordova JavaScript API calls to the *Android WebView* component that in turn make Android-API calls to the Android platform. Moreover, the Android WebView component delegates the Cordova JavaScript API calls to the responsible Cordova plugins. These plugins make Android API calls to the Android platform. Thus HTML Android Application can reach any native functionality either by preexisting or custom plugins. **2.62**

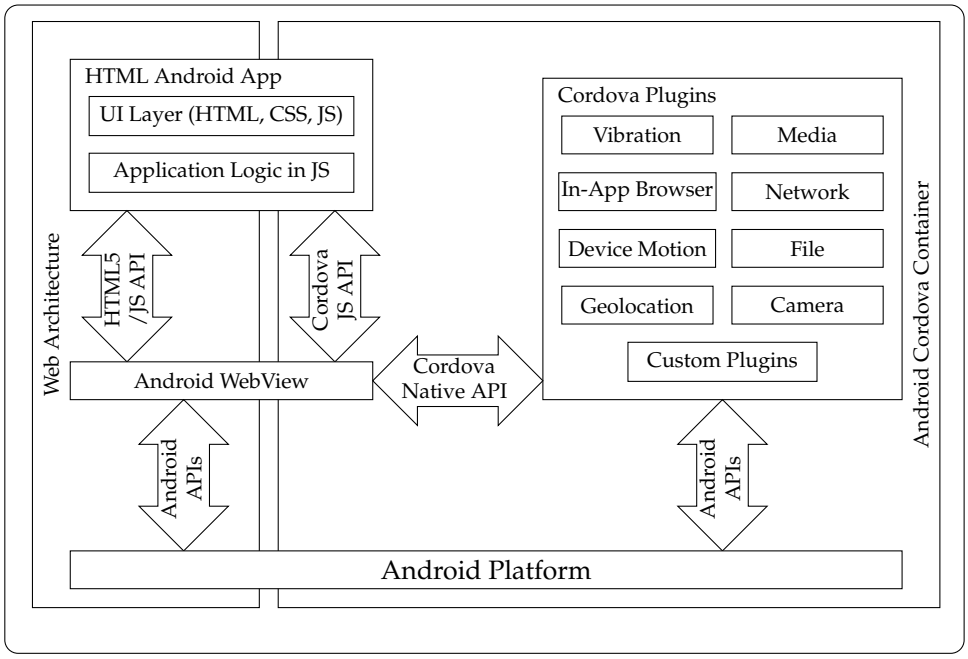


FIGURE 2.7: Architecture of an Apache Cordova application running on Android (cf. Brucker and Herzberg [BH16])

From a mobile application developer's perspective, the core of a Cordova application is provided by the HTML, CSS, and JavaScript code inside the folder *www*. The meta-information is stored in the *config.xml* file. These core artifacts are platform **2.63**

independent. Mobile application developers might add platform-specific code inside the folder *plugins*. The platform-independent program code is wrapped into a hybrid and platform-specific mobile application.

2.64 Even though the development of a cross-platform code generator is not intended by this work, a model-driven development infrastructure designer must keep in mind that the cross-platform approach might limit the architectural design of a mobile application. For example, different architectural patterns (e.g., Model-View-Controller) are not feasible since the cross-platform approach follows a flat architectural design by putting the application logic only to one single layer which in general seems not to be adequate.

2.65 As mentioned in the introduction, we focus on native application development due to their full capability in terms of hardware access. In particular, we contribute code generators for both Android and iOS platforms. However, using a model-driven development process, we are potentially able to provide code generators for cross-platforms (e.g., Apace Cordova).

2.4.5 Comparing Cross-Platform Technologies and MDD

2.66 A common characteristic of both approaches is the abstraction of the software platform. Cross-platform technologies reuse mostly web-based languages, whereas the model-driven development approach reuses existing general-purpose modeling languages or domain-specific modeling languages. From the developers' perspective, cross-platform approaches provide no abstractions in terms of technical concerns. Hence, cross-platform approaches are suitable for technical experts. The model-driven development approach might also be useful for domain experts with less technical skill. Moreover, cross-platform approaches are limited when it comes to accessing hardware. Such device-specific parts cannot be implemented in cross-platform languages, but can be realized using native languages.